# RECURSOS

## SESSIÓ 5 · FLUXOS, CSV AMB OPENCSV I ACCÉS ALEATORI

**Correspondència amb les RA:** RA1. Desenvolupa aplicacions que gestionen informació emmagatzemada en fitxers identificant-ne el camp d'aplicació i utilitzant classes específiques. CA 1.2, 1.3, 1.4 i 1.6.

Dimarts 29/09/2026 · DAM2 · sessió 5 de 58 · AEA1 «Gestió de fitxers amb Java» · AA2 «Llegeix i escriu dades» (2a part, lliurament)

### La sessió d'avui en un cop d'ull

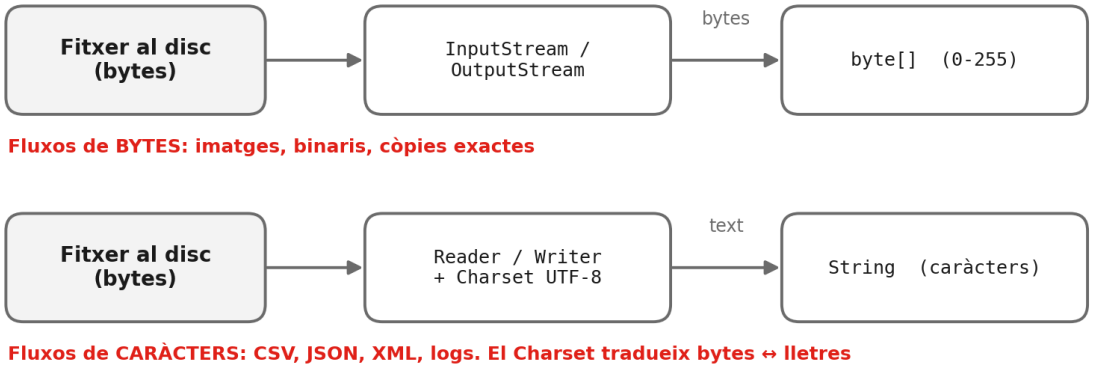
A la sessió 4 vam obrir els fitxers per dins: fluxos de bytes i de caràcters, `try-with-resources` i excepcions d'E/S. Avui tanquem l'AA2 amb les dues peces que faltaven: llegir i escriure CSV de veritat amb la biblioteca **OpenCSV**, que sap tractar les cometes i els separadors dins del text, i l'**accés aleatori** amb `RandomAccessFile`, que permet canviar un sol camp d'un fitxer sense reescriure'l sencer. Acabarem mesurant per què el segon és tan ràpid.

Tots els exemples són al repositori del mòdul, paquet `cat.pratfp.gamevault.sessio5`: **github.com/ilopez000/0486-acces-a-dades**. Clona'l o fes `git pull` abans de començar.

> **El que t'has d'endur**
> Un fitxer és una tira de bytes. Els fluxos de caràcters hi posen una codificació al davant per convertir-los en text; els de bytes els deixen tal com són.
> OpenCSV llegeix i escriu files de CSV respectant cometes i separadors; `split(";")` no.
> Amb registres de mida fixa, la posició d'un camp es calcula: `seek(posició)` i s'escriu només allò.

### 1. Repàs: dues famílies de fluxos

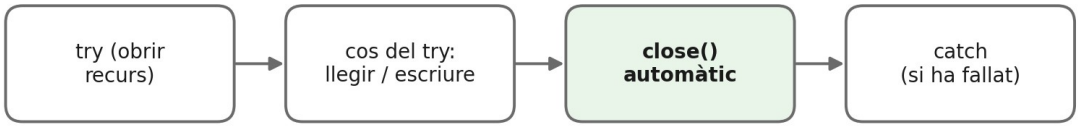
Java té dues maneres d'obrir un fitxer, i triar-ne una és la primera decisió de qualsevol programa d'accés a dades.




*Fluxos de bytes i fluxos de caràcters. La diferència és el Charset.*

| | Fluxos de bytes | Fluxos de caràcters |
|---|---|---|
| Classes base | `InputStream` / `OutputStream` | `Reader` / `Writer` |

| | Fluxos de bytes | Fluxos de caràcters |
|---|---|---|
| Unitat | byte (0-255) | caràcter (`char`, Unicode) |
| Per a què | imatges, àudio, fitxers binaris propis, còpies exactes | CSV, JSON, XML, logs, configuració |
| Com s'obren | `Files.newInputStream(p)` / `Files.newOutputStream(p)` | `Files.newBufferedReader(p, UTF_8)` / `newBufferedWriter(...)` |
| Decoradors útils | `BufferedInputStream`, `DataInputStream`, `ObjectInputStream` | `BufferedReader`, `PrintWriter`, `CSVReader` (OpenCSV) |




Passi el que passi (fins i tot amb una excepció), el close() s'executa sempre. Mai un close() manual.

*try-with-resources: el recurs es tanca sol, també si salta una excepció.*

**Compte amb això**
Si no indiques `StandardCharsets.UTF_8` en obrir un fitxer de text, Java fa servir la codificació del sistema. Al teu portàtil funcionarà i al del company sortiran símbols estranys als accents. Indica-la **sempre**.

## 2. Exemple 1 · Llegir text línia a línia i escriure un log en mode APPEND

Llegim el catàleg amb un `BufferedReader` i, en acabar, deixem constància al fitxer `dades/carrega.log`. El log s'obre en mode `APPEND`: cada execució hi afegeix una línia al final en lloc d'esborrar les anteriors.

**src/main/java/cat/pratfp/gamevault/sessio5/Ex1FluxosDeText.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import java.io.BufferedReader;
 4  import java.io.BufferedWriter;
 5  import java.io.IOException;
 6  import java.nio.charset.StandardCharsets;
 7  import java.nio.file.Files;
 8  import java.nio.file.Path;
 9  import java.nio.file.StandardOpenOption;
10  import java.time.LocalDateTime;
11
12  /** Exemple 1: llegir i escriure text línia a línia amb fluxos de caràcters.
*/
13  public class Ex1FluxosDeText {
14
15      public static void main(String[] args) throws IOException {
16          Path cataleg = Path.of("dades", "catalog.csv");
17          Path log = Path.of("dades", "carrega.log");
18
19          // 1. LLEGIR: BufferedReader llegeix línia a línia, amb la
codificació explícita.
20          int linies = 0;
21          try (BufferedReader lector = Files.newBufferedReader(cataleg,
StandardCharsets.UTF_8)) {
22              String linia;
23              while ((linia = lector.readLine()) != null) {
24                  linies++;
25                  if (linies == 1) continue;              // saltem la
capçalera
26                  String[] camps = linia.split(";");
27                  System.out.println(camps[1] + " -> " + camps[5] + " h");
28              }
29          }
30          System.out.println("Línies llegides: " + linies);
31
32          // 2. ESCRIURE EN MODE APPEND: cada execució afegeix una línia al
final del log.
33          try (BufferedWriter escriptor = Files.newBufferedWriter(log,
StandardCharsets.UTF_8,
34                  StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
35              escriptor.write(LocalDateTime.now() + " | catàleg llegit | " +
(linies - 1) + " jocs");
36              escriptor.newLine();
37          }
38          System.out.println("Log: " + Files.readString(log,
StandardCharsets.UTF_8).strip());
39      }
40  }
```

| Línia | Codi | Què fa i per què |
|---|---|---|
| **16** | `Path cataleg = Path.of("dades", "catalog.csv");` | Ruta relativa al projecte: `Path.of` posa el separador de carpetes correcte per a cada sistema operatiu. |
| **21** | `try (BufferedReader lector = Files.newBufferedReader(cata…` | `Files.newBufferedReader` obre el fitxer com a text, amb la codificació explícita. El `try (...)` el tancarà sol. |
| **22** | `String linia;` | La variable `linia` es declara fora del bucle perquè la condició del `while` la necessita. |

| Línia | Codi | Què fa i per què |
|---|---|---|
| **23** | `while ((linia = lector.readLine()) != null) {` | `readLine()` retorna la línia següent o `null` quan s'acaba el fitxer. Aquest és el patró clàssic de lectura. |
| **25** | `if (linies == 1) continue; // saltem la capç…` | La primera línia és la capçalera: `continue` salta a la iteració següent sense processar-la. |
| **26** | `String[] camps = linia.split(";");` | `split(";")` separa els camps. Funciona amb el catàleg net; amb un títol que porti punt i coma dins de cometes, no (ho resol OpenCSV a l'exemple 3). |
| **33-34** | `try (BufferedWriter escriptor = Files.newBufferedWriter(l…` | Obrim el log amb dues opcions: `CREATE` (si no existeix, es crea) i `APPEND` (s'escriu al final). Sense `APPEND`, cada execució el buidaria. |
| **35-36** | `escriptor.write(LocalDateTime.now() + " \| catàleg llegit …` | `write` escriu la cadena i `newLine()` posa el salt de línia propi del sistema. |
| **38** | `System.out.println("Log: " + Files.readString(log, Standa…` | `Files.readString` és la manera curta de llegir un fitxer sencer quan és petit. |

**Sortida esperada**

```
Hollow Knight -> 42.5 h
Blasphemous 2 -> 18.0 h
Metroid Dread -> 12.5 h
Gris -> 4.0 h
Rise of the Third Power -> 22.0 h
Línies llegides: 6
Log: 2026-09-29T15:12:41.562 | catàleg llegit | 5 jocs
```

Executa'l dues vegades i obre `dades/carrega.log`: hi trobaràs dues línies. Això és exactament el que ha de fer la classe `RegistreCarrega` de l'AA2.

## 3. Exemple 2 · Fluxos de bytes: copiar qualsevol fitxer i guardar números en binari

Quan el contingut no és text (o quan no volem interpretar-lo), treballem amb bytes. L'exemple fa dues coses: copia un fitxer a trossos de 8 KB, i escriu i llegeix un fitxer binari propi amb `DataOutputStream` i `DataInputStream`.

**src/main/java/cat/pratfp/gamevault/sessio5/Ex2FluxosDeBytes.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import java.io.BufferedInputStream;
 4  import java.io.BufferedOutputStream;
 5  import java.io.DataInputStream;
 6  import java.io.DataOutputStream;
 7  import java.io.IOException;
 8  import java.io.InputStream;
 9  import java.io.OutputStream;
10  import java.nio.file.Files;
11  import java.nio.file.Path;
12
13  /** Exemple 2: fluxos de bytes: copiar qualsevol fitxer i guardar dades
binàries. */
14  public class Ex2FluxosDeBytes {
15
16      private static final String CAPCALERA = "GAMEVAULT-STATS-v1";
17
18      public static void main(String[] args) throws IOException {
19          Path origen = Path.of("dades", "catalog.csv");
20          Path copia = Path.of("dades", "export", "catalog.copia");
21          Files.createDirectories(copia.getParent());
22
23          // 1. COPIAR BYTE A BYTE (amb memòria intermèdia): serveix per a
text, imatges, zips...
24          byte[] buffer = new byte[8 * 1024];
25          long total = 0;
26          try (InputStream in = new
BufferedInputStream(Files.newInputStream(origen));
27                OutputStream out = new
BufferedOutputStream(Files.newOutputStream(copia))) {
28              int llegits;
29              while ((llegits = in.read(buffer)) != -1) {
30                  out.write(buffer, 0, llegits);
31                  total += llegits;
32              }
33          }
34          System.out.println("Copiats " + total + " bytes -> " + copia);
35
36          // 2. ESCRIURE TIPUS PRIMITIUS: DataOutputStream guarda int i double
en binari.
37          Path stats = Path.of("dades", "export", "stats.bin");
38          try (DataOutputStream out = new DataOutputStream(
39                  new BufferedOutputStream(Files.newOutputStream(stats)))) {
40              out.writeUTF(CAPCALERA);   // una marca per reconèixer el fitxer
41              out.writeInt(5);           // total de jocs
42              out.writeDouble(99.0);     // hores totals
43          }
44          System.out.println("stats.bin ocupa " + Files.size(stats) + "
bytes");
45
46          // 3. LLEGIR-LOS EN EL MATEIX ORDRE i validar la capçalera.
47          try (DataInputStream in = new DataInputStream(
48                  new BufferedInputStream(Files.newInputStream(stats)))) {
49              String marca = in.readUTF();
50              if (!marca.equals(CAPCALERA)) {
51                  throw new IOException("Això no és un fitxer d'estadístiques:
" + marca);
52              }
53              System.out.println("Jocs: " + in.readInt() + " · Hores: " +
in.readDouble());
54          }
55      }
56  }
```

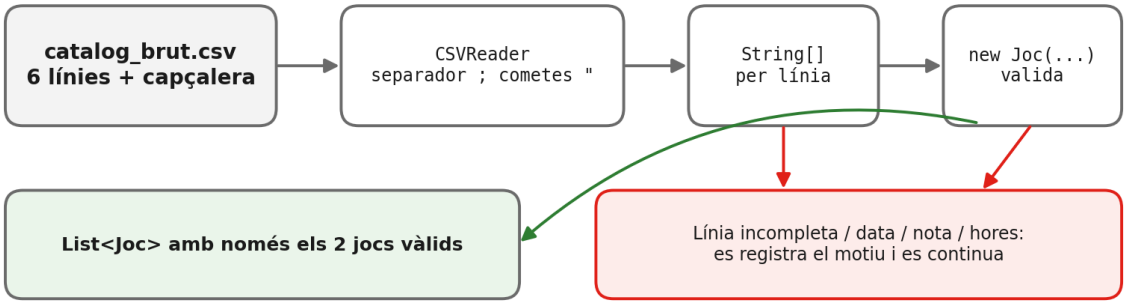
| Línia | Codi | Què fa i per què |
|---|---|---|
| **16** | `private static final String CAPCALERA = "GAMEVAULT-STATS-…` | Una constant de text que farem servir de «signatura» del fitxer binari: així sabrem si el que llegim és nostre. |
| **21** | `Files.createDirectories(copia.getParent());` | `createDirectories` crea `dades/export` si no existeix (i no falla si ja hi és). |
| **24** | `byte[] buffer = new byte[8 * 1024];` | La memòria intermèdia: llegim i escrivim de 8.192 bytes en 8.192, en lloc de byte a byte. És la diferència entre lent i ràpid. |
| **26-27** | `try (InputStream in = new BufferedInputStream(Files.newIn…` | Obrim els dos fluxos al mateix `try`: `Buffered...` embolcalla el flux bàsic i hi afegeix memòria intermèdia. Es tancaran en ordre invers. |
| **29-30** | `while ((llegits = in.read(buffer)) != -1) { …` | `read(buffer)` omple el buffer i retorna quants bytes ha llegit, o `-1` al final. `write(buffer, 0, llegits)` escriu **només** els llegits: l'última lectura sol ser més curta. |
| **38-39** | `try (DataOutputStream out = new DataOutputStream( …` | `DataOutputStream` sap escriure tipus primitius de Java en binari. Els «apilem»: fitxer → buffer → dades. |
| **40-42** | `out.writeUTF(CAPCALERA); // una marca per reconèixer el…` | Escrivim la signatura (`writeUTF`), un `int` (4 bytes) i un `double` (8 bytes). El fitxer resultant ocupa 32 bytes: 2 + 18 de la cadena, més 4, més 8. |
| **47-52** | `try (DataInputStream in = new DataInputStream( …` | Es llegeix en el **mateix ordre** que s'ha escrit. Si el primer que llegim no és la signatura, el fitxer no és nostre i llancem `IOException`. |
| **53** | `System.out.println("Jocs: " + in.readInt() + " · Hores: "…` | `readInt` i `readDouble` recuperen els valors exactes, sense passar per text. |

> **Sortida esperada**
>
> ```
> Copiats 329 bytes -> dades/export/catalog.copia
> stats.bin ocupa 32 bytes
> Jocs: 5 · Hores: 99.0
> ```

> **Relació amb l'AA2**
>
> Aquest exemple és el model de `MagatzemBinari`: `copiaCaratula` és el punt 1 (canvia `catalog.csv` per una imatge PNG) i `desaEstadistiques` / `llegeixEstadistiques` són el punt 2, amb la capçalera `GAMEVAULT-STATS-v1`.

## 4. Exemple 3 · OpenCSV: llegir un CSV brut sense que el programa peti

El CSV que arriba de fora mai és net. Tres línies de les sis tenen errors diferents, i una té un punt i coma dins del títol. `split` no se'n sortiria; OpenCSV sí. La regla d'or: **una línia dolenta no atura la càrrega**, es registra el motiu i es continua.




*Cada línia passa pel parser i per la validació del record Joc; les que fallen no entren a la llista.*

Per fer servir OpenCSV cal declarar-lo al `pom.xml` (IntelliJ el descarrega sol la primera vegada):

**pom.xml (dins de <dependencies>)**

```
<dependency>
    <groupId>com.opencsv</groupId>
    <artifactId>opencsv</artifactId>
    <version>5.9</version>
</dependency>
```

**src/main/java/cat/pratfp/gamevault/sessio5/Ex3LlegeixCsvBrut.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import cat.pratfp.gamevault.model.Joc;
 4  import com.opencsv.CSVParser;
 5  import com.opencsv.CSVParserBuilder;
 6  import com.opencsv.CSVReader;
 7  import com.opencsv.CSVReaderBuilder;
 8  import com.opencsv.exceptions.CsvValidationException;
 9
10  import java.io.IOException;
11  import java.io.Reader;
12  import java.nio.charset.StandardCharsets;
13  import java.nio.file.Files;
14  import java.nio.file.Path;
15  import java.time.LocalDate;
16  import java.time.format.DateTimeParseException;
17  import java.util.ArrayList;
18  import java.util.List;
19
20  /** Exemple 3: llegir un CSV brut amb OpenCSV sense que el programa peti. */
21  public class Ex3LlegeixCsvBrut {
22
23      static final String CSV_BRUT = """
24              id;titol;plataforma;estudi;dataSortida;horesJugades;nota
25              1;Hollow Knight;PC;Team Cherry;2017-02-24;42.5;9.5
26              2;Celeste;PC;Maddy Makes Games;2018-01-25;15.0;12.0
27              3;Gris;PC
28              4;Hades;PC;Supergiant Games;25-09-2020;30.0;9.0
29              5;"Baldur's Gate 3; Deluxe";PC;Larian Studios;2023-08-
03;120.0;9.8
30              6;Metroid Dread;Switch;MercurySteam;2021-10-08;-5.0;9.0
31              """;
32
33      public static void main(String[] args) throws IOException {
34          Path brut = Path.of("dades", "import", "catalog_brut.csv");
35          Files.createDirectories(brut.getParent());
36          Files.writeString(brut, CSV_BRUT, StandardCharsets.UTF_8);
37
38          List<Joc> valids = llegeix(brut);
39          System.out.println("Jocs vàlids: " + valids.size());
40          valids.forEach(j -> System.out.println("  " + j.id() + " · " +
j.titol()));
41      }
42
43      static List<Joc> llegeix(Path p) throws IOException {
44          List<Joc> jocs = new ArrayList<>();
45          // El parser sap que el separador és ; i que les cometes protegen el
text.
46          CSVParser parser = new
CSVParserBuilder().withSeparator(';').withQuoteChar('"').build();
47
48          try (Reader r = Files.newBufferedReader(p, StandardCharsets.UTF_8);
49               CSVReader csv = new
CSVReaderBuilder(r).withCSVParser(parser).withSkipLines(1).build()) {
50
51              String[] c;
52              int linia = 1;
53              while ((c = csv.readNext()) != null) {
54                  linia++;
55                  if (c.length < 7) {
56                      System.out.println("Línia " + linia + " descartada:
incompleta (" + c.length + " camps)");
57                      continue;
58                  }
```

```
59                  try {
60                      jocs.add(new Joc(Integer.parseInt(c[0].trim()),
c[1].trim(), c[2].trim(), c[3].trim(),
61                              LocalDate.parse(c[4].trim()),
Double.parseDouble(c[5].trim()),
62                              Double.parseDouble(c[6].trim())));
63                  } catch (DateTimeParseException e) {
64                      System.out.println("Línia " + linia + " descartada: data
mal formada '" + c[4] + "'");
65                  } catch (NumberFormatException e) {
66                      System.out.println("Línia " + linia + " descartada:
número invàlid");
67                  } catch (IllegalArgumentException e) {
68                      System.out.println("Línia " + linia + " descartada: " +
e.getMessage());
69                  }
70              }
71          } catch (CsvValidationException e) {
72              throw new IOException("CSV mal format a " + p, e);
73          }
74          return jocs;
75      }
76  }
```

| Línia | Codi | Què fa i per què |
|---|---|---|
| **23-31** | `static final String CSV_BRUT = """ id;titol;p…` | Un «text block» (`"""`) amb el CSV brut de l'AA2, tal qual. Fixa't en la línia 5: el títol va entre cometes perquè conté un punt i coma. |
| **35-36** | `Files.createDirectories(brut.getParent()); Files.…` | L'exemple escriu el fitxer a `dades/import` perquè el puguis obrir; a l'AA2 el tindràs ja fet. |
| **43** | `static List<Joc> llegeix(Path p) throws IOException {` | El mètode `llegeix` és la peça que reutilitzaràs: rep una ruta i retorna només els jocs vàlids. |
| **46** | `CSVParser parser = new CSVParserBuilder().withSeparator('…` | `CSVParserBuilder` configura **com** es tallen les línies: separador `;` i cometes `"` per protegir text amb separadors dins. |
| **48-49** | `try (Reader r = Files.newBufferedReader(p, StandardCharse…` | Dos recursos al `try`: el `Reader` de text amb UTF-8 i, a sobre, el `CSVReader` amb el nostre parser. `withSkipLines(1)` salta la capçalera. |
| **53** | `while ((c = csv.readNext()) != null) {` | `readNext()` retorna la fila següent com a `String[]`, ja tallada, o `null` al final. |
| **55-58** | `if (c.length < 7) { System.out.printl…` | Primera comprovació: si falten camps, la línia és incompleta. S'informa i `continue` passa a la següent. Cap excepció. |
| **59-63** | `try { jocs.add(new Joc(Integer.parseI…` | Es construeix el `Joc`. `Integer.parseInt`, `LocalDate.parse` i `Double.parseDouble` converteixen text a valors; `trim()` treu espais sobrants. |
| **64-65** | `System.out.println("Línia " + linia + " descartada: data …` | Data amb format incorrecte (`25-09-2020`): `LocalDate.parse` llança `DateTimeParseException`. La capturem **abans** que les altres, perquè és més específica. |
| **66-67** | `System.out.println("Línia " + linia + " descartada: númer…` | Un número que no ho és: `NumberFormatException`. |

| Línia | Codi | Què fa i per què |
|---|---|---|
| **68-69** | `System.out.println("Línia " + linia + " descartada: " + e…` | La validació del constructor compacte de `Joc` (nota fora de rang, hores negatives) arriba com a `IllegalArgumentException` amb el missatge que hi vam posar. |
| **72-73** | `throw new IOException("CSV mal format a " + p, e);`<br>`}` | `CsvValidationException` és un error d'estructura del CSV sencer, no d'una línia: aquí sí que aturem la càrrega, embolicant-la en una `IOException` amb un missatge útil i **sense perdre la causa**. |

**Sortida esperada**

```
Línia 3 descartada: la nota ha d'anar de 0 a 10: 12.0
Línia 4 descartada: incompleta (3 camps)
Línia 5 descartada: data mal formada '25-09-2020'
Línia 7 descartada: les hores no poden ser negatives: -5.0
Jocs vàlids: 2
  1 · Hollow Knight
  5 · Baldur's Gate 3; Deluxe
```

Observa que el joc 5 ha entrat amb el punt i coma dins del títol: el parser ha entès les cometes. A l'AA2, en lloc de `System.out.println`, cada descart va a `RegistreCarrega` amb data i hora.

## 5. Exemple 4 · OpenCSV: escriure el catàleg i afegir-hi un joc

Escriure té dos modes que l'AA2 demana per separat: **sobreescriure** el fitxer sencer (amb capçalera) i **afegir** una fila al final (sense repetir-la).

**src/main/java/cat/pratfp/gamevault/sessio5/Ex4EscriuCsv.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import cat.pratfp.gamevault.model.Joc;
 4  import com.opencsv.CSVWriter;
 5  import com.opencsv.ICSVWriter;
 6
 7  import java.io.IOException;
 8  import java.io.Writer;
 9  import java.nio.charset.StandardCharsets;
10  import java.nio.file.Files;
11  import java.nio.file.Path;
12  import java.nio.file.StandardOpenOption;
13  import java.time.LocalDate;
14  import java.util.List;
15
16  /** Exemple 4: escriure un catàleg sencer i afegir-hi un joc al final amb
OpenCSV. */
17  public class Ex4EscriuCsv {
18
19      private static final String[] CAPCALERA =
20              {"id", "titol", "plataforma", "estudi", "dataSortida",
"horesJugades", "nota"};
21
22      public static void main(String[] args) throws IOException {
23          Path desti = Path.of("dades", "export", "cataleg_nou.csv");
24          Files.createDirectories(desti.getParent());
25
26          List<Joc> jocs = List.of(
27                  new Joc(1, "Hollow Knight", "PC", "Team Cherry",
LocalDate.of(2017, 2, 24), 42.5, 9.5),
28                  new Joc(2, "Baldur's Gate 3; Deluxe", "PC", "Larian
Studios", LocalDate.of(2023, 8, 3), 120.0, 9.8));
29
30          // 1. ESCRIURE-HO TOT: sobreescriu el fitxer i posa la capçalera.
31          escriu(desti, jocs);
32
33          // 2. AFEGIR UN JOC: mode APPEND, sense repetir la capçalera.
34          afegeix(desti, new Joc(3, "Gris", "PC", "Nomada Studio",
LocalDate.of(2018, 12, 13), 4.0, 8.0));
35
36          System.out.print(Files.readString(desti, StandardCharsets.UTF_8));
37      }
38
39      static void escriu(Path p, List<Joc> jocs) throws IOException {
40          try (Writer w = Files.newBufferedWriter(p, StandardCharsets.UTF_8);
41               CSVWriter csv = new CSVWriter(w, ';',
ICSVWriter.DEFAULT_QUOTE_CHARACTER,
42                       ICSVWriter.DEFAULT_ESCAPE_CHARACTER,
ICSVWriter.DEFAULT_LINE_END)) {
43              csv.writeNext(CAPCALERA, false);      // false = sense cometes
si no calen
44              for (Joc j : jocs) {
45                  csv.writeNext(aCamps(j), false);
46              }
47          }
48      }
49
50      static void afegeix(Path p, Joc j) throws IOException {
51          try (Writer w = Files.newBufferedWriter(p, StandardCharsets.UTF_8,
StandardOpenOption.APPEND);
52               CSVWriter csv = new CSVWriter(w, ';',
ICSVWriter.DEFAULT_QUOTE_CHARACTER,
53                       ICSVWriter.DEFAULT_ESCAPE_CHARACTER,
ICSVWriter.DEFAULT_LINE_END)) {
```

```
54              csv.writeNext(aCamps(j), false);
55          }
56      }
57
58      /** Converteix un Joc en la fila de text que espera el CSV. */
59      static String[] aCamps(Joc j) {
60          return new String[]{String.valueOf(j.id()), j.titol(),
j.plataforma(), j.estudi(),
61                  j.dataSortida().toString(),
String.valueOf(j.horesJugades()), String.valueOf(j.nota())};
62      }
63  }
```

| Línia | Codi | Què fa i per què |
|---|---|---|
| **19-20** | `private static final String[] CAPCALERA = {"i…` | La capçalera com a constant: així no es pot escriure diferent en dos llocs. |
| **26-28** | `List<Joc> jocs = List.of( new Joc(1, "Hol…` | Dos jocs de mostra. El segon porta un punt i coma al títol expressament, per veure què en fa el `CSVWriter`. |
| **31** | `escriu(desti, jocs);` | Primer mode: escriure-ho tot. Si el fitxer existia, es reemplaça. |
| **34** | `afegeix(desti, new Joc(3, "Gris", "PC", "Nomada Studio", …` | Segon mode: afegir un sol joc al final. |
| **40-42** | `try (Writer w = Files.newBufferedWriter(p, StandardCharse…` | `CSVWriter` es construeix sobre un `Writer` de text (UTF-8). Els paràmetres són: separador `;`, caràcter de cometes, caràcter d'escapament i final de línia. Fem servir els valors per defecte d'`ICSVWriter` per als tres últims. |
| **43** | `csv.writeNext(CAPCALERA, false); // false = sense co…` | `writeNext(camps, false)`: el `false` vol dir «no posis cometes a tot». OpenCSV les posarà **només** on calgui (quan un camp conté el separador). |
| **44-46** | `for (Joc j : jocs) { csv.writeNext(aCamps…` | Una fila per joc. `aCamps` converteix el `Joc` en `String[]`. |
| **51** | `try (Writer w = Files.newBufferedWriter(p, StandardCharse…` | Per afegir, s'obre el `Writer` amb `StandardOpenOption.APPEND`. La resta és idèntica, però sense escriure la capçalera. |
| **59-61** | `static String[] aCamps(Joc j) { return new String…` | La conversió a text de cada camp: `String.valueOf` per als números, `toString()` per a la data (dona el format ISO `2017-02-24`). |

**Sortida esperada (contingut de dades/export/cataleg_nou.csv)**

```
id;titol;plataforma;estudi;dataSortida;horesJugades;nota
1;Hollow Knight;PC;Team Cherry;2017-02-24;42.5;9.5
2;"Baldur's Gate 3; Deluxe";PC;Larian Studios;2023-08-03;120.0;9.8
3;Gris;PC;Nomada Studio;2018-12-13;4.0;8.0
```

Prova d'anada i tornada: llegeix aquest fitxer amb el `llegeix` de l'exemple 3. Han de sortir 3 jocs, i el segon amb el títol sencer. És una de les cinc proves JUnit de l'AA2.

## 6. Exemple 5 · Accés aleatori: registres de mida fixa i seek

Fins ara, per canviar una dada calia reescriure tot el fitxer. `RandomAccessFile` permet situar-se en un byte concret i llegir o escriure just allà. La condició: saber **on** és cada dada, i per això tots els registres han de pesar el mateix.

**Un registre = 4 + 80 + 8 = 92 bytes, sempre igual**




Registre 0 comença al byte 0 · Registre 1 al 92 · Registre 2 al 184 · Registre n al n × 92

`Les hores del registre 2 són al byte 184 + 4 + 80 = 268 → seek(268); writeDouble(15.5)`

*Disseny del registre. Amb mida fixa, la posició de qualsevol camp es calcula amb una multiplicació.*

**src/main/java/cat/pratfp/gamevault/sessio5/Ex5AccesAleatori.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import java.io.IOException;
 4  import java.io.RandomAccessFile;
 5  import java.nio.file.Files;
 6  import java.nio.file.Path;
 7
 8  /** Exemple 5: registres de mida fixa amb RandomAccessFile i seek. */
 9  public class Ex5AccesAleatori {
10
11      // Cada registre: id (4 bytes) + títol de 40 chars (80 bytes) + hores (8
bytes) = 92 bytes.
12      static final int LONG_TITOL = 40;
13      static final int MIDA_REGISTRE = Integer.BYTES + LONG_TITOL *
Character.BYTES + Double.BYTES;
14
15      public static void main(String[] args) throws IOException {
16          Path fitxer = Path.of("dades", "export", "hores.dat");
17          Files.createDirectories(fitxer.getParent());
18          Files.deleteIfExists(fitxer);
19
20          String[] titols = {"Hollow Knight", "Blasphemous 2", "Metroid
Dread", "Gris", "Rise of the Third Power"};
21          double[] hores = {42.5, 18.0, 12.5, 4.0, 22.0};
22
23          // 1. ESCRIURE 5 REGISTRES, un darrere l'altre.
24          try (RandomAccessFile raf = new RandomAccessFile(fitxer.toFile(),
"rw")) {
25              for (int i = 0; i < titols.length; i++) {
26                  escriu(raf, i, i + 1, titols[i], hores[i]);
27              }
28          }
29          mostra(fitxer, "Abans");
30
31          // 2. ACTUALITZAR NOMÉS LES HORES DEL REGISTRE 2: seek directe al
camp.
32          try (RandomAccessFile raf = new RandomAccessFile(fitxer.toFile(),
"rw")) {
33              long posicioHores = 2L * MIDA_REGISTRE + Integer.BYTES +
LONG_TITOL * Character.BYTES;
34              raf.seek(posicioHores);
35              raf.writeDouble(12.5 + 3.0);   // hem jugat 3 hores més
36          }
37          mostra(fitxer, "Després");
38      }
39
40      static void escriu(RandomAccessFile raf, int posicio, int id, String
titol, double hores) throws IOException {
41          raf.seek((long) posicio * MIDA_REGISTRE);
42          raf.writeInt(id);
43          // El títol s'omple (o es talla) fins a 40 caràcters perquè tots els
registres pesin igual.
44          String fix = String.format("%-" + LONG_TITOL + "s",
titol).substring(0, LONG_TITOL);
45          raf.writeChars(fix);
46          raf.writeDouble(hores);
47      }
48
49      static void mostra(Path fitxer, String etiqueta) throws IOException {
50          System.out.println("--- " + etiqueta + " (" + Files.size(fitxer) /
MIDA_REGISTRE + " registres) ---");
51          try (RandomAccessFile raf = new RandomAccessFile(fitxer.toFile(),
"r")) {
52              long registres = raf.length() / MIDA_REGISTRE;
```

```
53              for (int i = 0; i < registres; i++) {
54                  raf.seek((long) i * MIDA_REGISTRE);
55                  int id = raf.readInt();
56                  StringBuilder titol = new StringBuilder();
57                  for (int c = 0; c < LONG_TITOL; c++)
titol.append(raf.readChar());
58                  double h = raf.readDouble();
59                  System.out.printf("%d | %-25s | %5.1f h%n", id,
titol.toString().strip(), h);
60              }
61          }
62      }
63  }
```

| Línia | Codi | Què fa i per què |
|---|---|---|
| **12-13** | `static final int LONG_TITOL = 40;`<br>`static final int MI…` | La mida del registre es **calcula** a partir de les constants de Java: `Integer.BYTES` (4), `Character.BYTES` (2, perquè `writeChars` escriu 2 bytes per caràcter) i `Double.BYTES` (8). Total 92. |
| **18** | `Files.deleteIfExists(fitxer);` | Comencem de zero: si el fitxer existia, l'esborrem perquè els registres vells no es barregin. |
| **24-28** | `try (RandomAccessFile raf = new RandomAccessFile(fitxer.t…` | Obrim el fitxer en mode `"rw"` (lectura i escriptura) i hi escrivim 5 registres a les posicions 0..4. |
| **32-36** | `try (RandomAccessFile raf = new RandomAccessFile(fitxer.t…` | La part important: calculem la posició del camp `hores` del registre 2 (2 × 92 + 4 + 80 = 268), hi saltem amb `seek` i escrivim **només** el `double`. Cap altre byte del fitxer canvia. |
| **41** | `raf.seek((long) posicio * MIDA_REGISTRE);` | Cada registre comença a `posicio × MIDA_REGISTRE`. `seek` hi mou el cursor. |
| **42** | `raf.writeInt(id);` | `writeInt` escriu els 4 bytes de l'id. |
| **44-45** | `String fix = String.format("%-" + LONG_TITOL + "s", titol…` | El títol s'omple amb espais fins a 40 caràcters (`%-40s`) o es talla si és més llarg (`substring`). Sense això, els registres no pesarien igual. |
| **45** | `raf.writeChars(fix);` | `writeChars` escriu cada caràcter en 2 bytes: 80 bytes en total. |
| **46** | `raf.writeDouble(hores);` | `writeDouble` escriu els 8 bytes de les hores. |
| **52** | `long registres = raf.length() / MIDA_REGISTRE;` | El nombre de registres es dedueix de la mida del fitxer dividida per la mida del registre. |
| **53-59** | `for (int i = 0; i < registres; i++) {`<br>`raf…` | Per mostrar-ho, recorrem els registres en ordre: `seek` a cadascun i lectura en el **mateix ordre** que l'escriptura: `readInt`, 40 × `readChar`, `readDouble`. `strip()` treu els espais de farciment. |

**Sortida esperada**

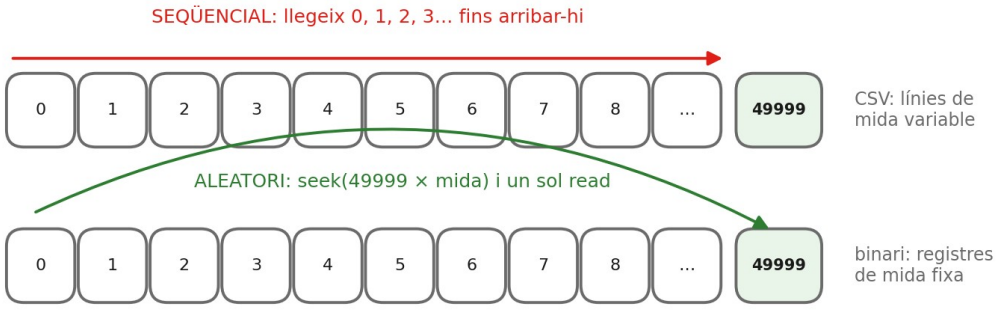
```
--- Abans (5 registres) ---
1 | Hollow Knight              |  42.5 h
2 | Blasphemous 2              |  18.0 h
3 | Metroid Dread              |  12.5 h
4 | Gris                       |   4.0 h
5 | Rise of the Third Power    |  22.0 h
--- Després (5 registres) ---
1 | Hollow Knight              |  42.5 h
2 | Blasphemous 2              |  18.0 h
3 | Metroid Dread              |  15.5 h
4 | Gris                       |   4.0 h
5 | Rise of the Third Power    |  22.0 h
```

**Compte amb això**

`writeChars` gasta **2 bytes per caràcter**. Si calcules la mida del registre pensant en 1 byte per lletra, tots els seek quedaran desplaçats i llegiràs brossa. A l'AA2, `actualitzaHores` ha de fer exactament el que fan les línies 32-36.

## 7. Exemple 6 · Mesura-ho: seqüencial contra aleatori (CA 1.2)

L'AA2 demana un argument, no una opinió: quant triga cada forma d'accés a arribar a l'últim de 50.000 registres. Generem els dos fitxers i cronometrem amb `System.nanoTime()`.




*Per arribar al registre 49.999, l'accés seqüencial passa per tots els anteriors; l'aleatori hi salta.*

**src/main/java/cat/pratfp/gamevault/sessio5/Ex6SequencialVsAleatori.java**

```
 1  package cat.pratfp.gamevault.sessio5;
 2
 3  import java.io.BufferedReader;
 4  import java.io.BufferedWriter;
 5  import java.io.IOException;
 6  import java.io.RandomAccessFile;
 7  import java.nio.charset.StandardCharsets;
 8  import java.nio.file.Files;
 9  import java.nio.file.Path;
10
11  /** Exemple 6: quant costa arribar a l'últim registre, seqüencialment i amb
seek. */
12  public class Ex6SequencialVsAleatori {
13
14      static final int N = 50_000;
15      static final int MIDA_REGISTRE = Integer.BYTES + Double.BYTES;   // id +
hores
16
17      public static void main(String[] args) throws IOException {
18          Path csv = Path.of("dades", "export", "gran.csv");
19          Path bin = Path.of("dades", "export", "gran.dat");
20          Files.createDirectories(csv.getParent());
21
22          // 1. GENERAR 50.000 registres sintètics en els dos formats.
23          try (BufferedWriter w = Files.newBufferedWriter(csv,
StandardCharsets.UTF_8);
24               RandomAccessFile raf = new RandomAccessFile(bin.toFile(),
"rw")) {
25              raf.setLength(0);
26              for (int i = 0; i < N; i++) {
27                  w.write(i + ";Joc " + i + ";" + (i % 100) + ".0");
28                  w.newLine();
29                  raf.writeInt(i);
30                  raf.writeDouble(i % 100);
31              }
32          }
33
34          // 2. SEQÜENCIAL: cal passar per les 49.999 línies anteriors.
35          long t0 = System.nanoTime();
36          String ultima = null;
37          try (BufferedReader r = Files.newBufferedReader(csv,
StandardCharsets.UTF_8)) {
38              String l;
39              while ((l = r.readLine()) != null) ultima = l;
40          }
41          long tSeq = System.nanoTime() - t0;
42
43          // 3. ALEATORI: un sol salt calculat.
44          t0 = System.nanoTime();
45          int id;
46          double hores;
47          try (RandomAccessFile raf = new RandomAccessFile(bin.toFile(), "r"))
{
48              raf.seek((long) (N - 1) * MIDA_REGISTRE);
49              id = raf.readInt();
50              hores = raf.readDouble();
51          }
52          long tAle = System.nanoTime() - t0;
53
54          System.out.println("Seqüencial: " + ultima + "  en " + tSeq / 1_000
+ " µs");
55          System.out.println("Aleatori:   " + id + ";" + hores + "  en " +
tAle / 1_000 + " µs");
56          System.out.println("El salt directe ha estat " + (tSeq /
Math.max(tAle, 1)) + " vegades més ràpid");
```

| Línia | Codi | Què fa i per què |
|---|---|---|
| **14-15** | `static final int N = 50_000;` `static final int MIDA_RE…` | 50.000 registres. El registre binari només té id i hores (12 bytes) per simplificar. |
| **23-32** | `try (BufferedWriter w = Files.newBufferedWriter(csv, Stan…` | Un sol bucle escriu les dues versions: la línia de text al CSV i els 12 bytes al fitxer binari. `setLength(0)` buida el binari si ja existia. |
| **35-41** | `long t0 = System.nanoTime();` `String ultima = null…` | Cronòmetre en marxa, lectura de **totes** les línies fins a la darrera, cronòmetre aturat. La diferència en nanosegons és el temps seqüencial. |
| **44-52** | `t0 = System.nanoTime();` `int id;` `double ho…` | El mateix amb accés aleatori: un seek calculat (`(N-1) × 12`) i dues lectures. Res més. |
| **54-56** | `System.out.println("Seqüencial: " + ultima + "  en " + tS…` | Els temps es mostren en microsegons (`/ 1_000`). `Math.max(tAle, 1)` evita una divisió per zero si el salt ha estat instantani. |

> **Sortida esperada (els temps varien segons l'ordinador)**
>
> ```
> Seqüencial: 49999;Joc 49999;99.0  en 30595 µs
> Aleatori:   49999;99.0  en 657 µs
> El salt directe ha estat 46 vegades més ràpid
> ```

Executa'l tres vegades i anota la mediana: la primera execució sol ser més lenta perquè el sistema encara no té el fitxer a la memòria cau. Aquests són els números que van a `ANALISI.md`, amb la teva explicació de **quan** compensa cada opció (pista: què passa si el que vols és llegir-ho tot?).

## 8. Resum de classes i mètodes d'avui

| Classe | Mètodes clau | Quan la fas servir |
|---|---|---|
| `BufferedReader` / `BufferedWriter` | `readLine()`, `write()`, `newLine()` | Text línia a línia. Sempre amb `StandardCharsets.UTF_8`. |
| `StandardOpenOption` | `CREATE`, `APPEND`, `TRUNCATE_EXISTING` | Per decidir si s'afegeix o se sobreescriu. |
| `BufferedInputStream` / `BufferedOutputStream` | `read(byte[])`, `write(byte[], 0, n)` | Copiar fitxers de qualsevol tipus, a trossos. |
| `DataOutputStream` / `DataInputStream` | `writeUTF`, `writeInt`, `writeDouble` i els `read...` | Fitxers binaris propis amb tipus primitius. Mateix ordre en llegir. |
| `CSVReader` + `CSVParserBuilder` | `withSeparator`, `withQuoteChar`, `withSkipLines`, `readNext()` | Llegir CSV respectant cometes. Retorna `String[]`. |
| `CSVWriter` | `writeNext(String[], boolean)` | Escriure CSV. Posa cometes només on cal. |
| `RandomAccessFile` | `seek`, `length`, `writeInt/Chars/Double`, `readInt/Char/Double` | Modificar un camp concret sense reescriure el fitxer. |
| `System.nanoTime()` | — | Cronometrar trossos de codi (diferència entre dues crides). |

## 9. Relació amb l'AA2 «Llegeix i escriu dades»

| Punt de l'AA2 | Classe que has de fer | Exemple que t'hi ajuda |
|---|---|---|
| 1 · OpenCSV al pom.xml | — | Apartat 4 |
| 2 · Model `Joc` amb validació | `cat.pratfp.gamevault.model.Joc` | Ja és al repositori (paquet `model`) |
| 3 · `llegeix`, `escriu`, `afegeix` | `CsvJocDao` | Exemples 3 i 4 |
| 4 · Càrrega del CSV brut | — | Exemple 3: 2 vàlids i 4 incidències |
| 5 · Registre d'errors amb data i hora | `RegistreCarrega` | Exemple 1 (mode APPEND) |
| 6 · Còpia de caràtula i estadístiques | `MagatzemBinari` | Exemple 2 |
| 7 · Registres de mida fixa i `actualitzaHores` | `MagatzemAleatori` | Exemple 5 |
| 8 · Mesura seqüencial vs aleatori | `ANALISI.md` | Exemple 6 |
| 9 · `CatalegException` amb causa | capa de servei | Exemple 3, línies 72-73 (el patró de «embolicar» una excepció) |

**Lliurament de l'AA2:** avui, 29/09, a les 23:59, al Classroom: enllaç al repositori i PDF `AA2_AEA1_0486_Cognom1Cognom2_Nom.pdf`. Abans de lliurar, revisió creuada per parelles amb la llista de comprovació de la fitxa.

## 10. Problemes típics i solució

| Símptoma | Causa probable | Solució |
|---|---|---|
| Els accents surten com Ã© o ? | Fitxer obert sense indicar la codificació | Afegeix `StandardCharsets.UTF_8` a `newBufferedReader` / `newBufferedWriter`. |
| El log només té l'última línia | Falta `StandardOpenOption.APPEND` | Obre el `Writer` amb `CREATE` i `APPEND`. |
| «Baldur's Gate 3; Deluxe» surt partit en dos camps | Estàs fent `split(";")` | Fes servir `CSVReader` amb `withQuoteChar('"')`. |
| `package com.opencsv does not exist` | Dependència no descarregada | Comprova el `pom.xml` i fes *Reload Maven project* a IntelliJ. |
| `RandomAccessFile` llegeix caràcters estranys | Mida de registre mal calculada | Recorda: `writeChars` són 2 bytes per caràcter. Usa `Character.BYTES`. |
| `EOFException` en llegir el binari | Llegeixes en un ordre diferent del d'escriptura, o més del que hi ha | Mateix ordre i mateixos tipus que en escriure; comprova `length()` abans de seek. |
| Temps aleatori 0 µs o negatiu | El sistema té el fitxer a la cau i el seek és instantani | És normal; executa-ho diverses vegades i mira la mediana. |

## 11. Comprova que ho has entès

1. Per què `Files.newBufferedReader(p)` sense `Charset` és un error encara que «funcioni» al teu ordinador?
2. Quina diferència hi ha entre obrir un `Writer` amb `APPEND` i sense?
3. Al CSV brut, per què la línia de «Baldur's Gate 3; Deluxe» té 7 camps i no 8?
4. Si el títol tingués 60 caràcters en lloc de 40, què passaria amb els registres de l'exemple 5?
5. A quin byte comencen les hores del registre 4 (el cinquè) de l'exemple 5?
6. Per què l'accés aleatori no serveix per a un CSV normal?

### Solucions

- 1. Perquè agafa la codificació per defecte del sistema, que canvia d'un ordinador a un altre. El mateix fitxer es llegiria diferent.
- 2. Amb `APPEND` s'escriu al final del que hi ha; sense, el fitxer es buida (`TRUNCATE_EXISTING`) i s'escriu de zero.
- 3. Perquè el punt i coma va entre cometes i el parser sap que forma part del camp, no que el separa.
- 4. `substring(0, 40)` el tallaria a 40 caràcters. La mida del registre no canvia; el títol perdria lletres.
- 5. $4 \times 92 + 4 + 80 = 452$.
- 6. Perquè les línies tenen mida variable: no es pot calcular on comença la línia n sense llegir les anteriors.

## 12. Per a la propera sessió

Dimecres 30/09/2026 · AA3 «Canvia de format» (1a part): serialització d'objectes amb `ObjectOutputStream`, els riscos que té, i JSON amb Jackson. Llegeix abans l'apartat 5 dels apunts de l'AEA1 i la pàgina d'inici de la documentació de Jackson (github.com/FasterXML/jackson-databind).

## Bibliografia/Webgrafia


- Oracle. Java SE 21 API: `java.io` i `java.nio.file`. docs.oracle.com/en/java/javase/21/docs/api/
- Oracle. Learn Java: Java I/O. dev.java/learn/java-io/
- OpenCSV 5.9. User guide. opencsv.sourceforge.net
- Oracle. The Java Tutorials: Random Access Files. docs.oracle.com/javase/tutorial/essential/io/rafs.html
- Bloch, J. Effective Java (3a ed.), ítem 9: «Prefer try-with-resources to try-finally».
- Material del mòdul: apunts AEA1 «Gestió de fitxers amb Java», apartats 2, 3 i 4, i fitxa de l'AA2 (Classroom).
- Repositori d'exemples: github.com/ilopez000/0486-acces-a-dades


**Data actualització i autor/res**

Ignasi López Aylagas · 29/09/2026